EXPERIMENT 3

ROLL NO:240701567
NAME:THENDRAL V

Design and implement a To-Do List application with CLI, GUI, and VUI interfaces. Analyse the effect of interaction modality on user experience, learnability, and cognitive load.

Familiar and Unfamiliar Interaction in UI/UX Design

Familiar Interaction refers to standard patterns users already know — menu-driven inputs, button clicks, and text entry fields. These require little or no learning curve.

Eg: Numbered menus (CLI), Add/Remove buttons (GUI), simple voice commands like 'add task' or 'exit' (VUI).

Unfamiliar Interaction includes novel patterns users may not expect — such as keyword-flexible voice parsing, speech synthesis feedback, or dynamic status labels.

Eg: Natural language number parsing in VUI ('two' → 2), spoken error recovery, GUI key-binding (Enter to Add).

FRAME 1:

CLI (Command-Line Interface):

The CLI To-Do List is a terminal-based application where the user navigates through a numbered menu by typing choices. It uses only Python built-in I/O — no external libraries required.

Familiar Navigation Elements

- A numbered menu (1–4) listing all available actions at every loop iteration — a standard terminal interaction pattern.
- A sequential prompt flow: choose action → provide input → receive confirmation — familiar from any CLI tool.
- Error messages on invalid input guide the user back without crashing the application.

Unfamiliar Navigation Elements

- There are no visual affordances; the user must memorise or re-read option numbers each time.
- The absence of persistent task display means users must actively request to view tasks, which differs from GUI expectations.

Chunking Analysis

Sequential Menu Chunking

- All four options are printed on one line, grouping navigation into a single cognitive unit and reducing decision overhead.
- Numeric shortcuts (1, 2, 3, 4) minimise keystrokes and leverage positional memory.

Task Display Chunking

- Tasks are listed with incremental numbers that directly map to the remove-task input, eliminating translation effort between viewing and acting.

Error Handling as Safeguard

- A try/except block around integer parsing prevents loop breakage, and a clear 'Invalid choice' message reanchors the user.

FRAME 2:

GUI (Graphical User Interface):

The GUI To-Do List uses Python's Tkinter library to present a windowed desktop application. Users interact through labelled buttons, an entry field, and a scrollable task listbox. A status bar provides real-time feedback after every action.

Familiar Navigation Elements

- A text entry field with an Add button — a standard input pattern found in every task or note-taking app.
- A scrollable list displaying tasks with index numbers — immediately recognisable to any desktop user.
- Remove Selected and Clear All buttons with confirmation dialog — familiar destructive-action patterns.

Unfamiliar Navigation Elements

- The Return key binding on the entry field (adds task on Enter) may not be expected by users who rely on mouse-only interaction.
- The dynamic status bar at the bottom changes colour (green/red) based on action type — a subtle feedback cue that some users may overlook.

Chunking Analysis

Spatial Chunking via Visual Zones

- The window is divided into four vertical zones — Title, Input, List, and Action — each grouping only its related components.
- This spatial layout reduces visual search time; users know intuitively where to look for each interaction type.

Colour-Coded Action Chunking

- Blue = Add (constructive), Red = Remove (destructive), Grey = Clear All (neutral). Colour communicates intent without requiring text reading.

Feedback Chunking

- The status bar consolidates all feedback into one persistent location, avoiding visual clutter elsewhere on screen.

FRAME 3:

VUI (Voice User Interface):

The VUI To-Do List replaces all visual and tactile interaction with speech. It uses the SpeechRecognition library to capture microphone input and pyttsx3 for text-to-speech output. The full interaction loop — navigation, task dictation, list playback, and removal — happens through spoken language.

Familiar Navigation Elements

- Natural language commands ('add task', 'view tasks', 'exit') mirror how people naturally speak to voice assistants.

- The system restates all options at the start of every turn, functioning like a spoken menu — familiar from telephone IVR systems.
- Task confirmation is spoken aloud immediately after adding or removing, matching expected feedback behaviour.

Unfamiliar Navigation Elements

- Spoken number-word parsing ('two' → 2) is a non-obvious feature that users must discover through use or documentation.
- The absence of any visual display means users cannot refer back to previously heard content — a significant departure from GUI expectations.
- Ambient noise sensitivity and internet dependency (Google Speech API) are constraints most users do not anticipate.

Chunking Analysis

Auditory Menu Chunking

- All four options are grouped into a single spoken sentence, keeping them within auditory working memory (Miller's 7 ± 2 limit).
- Keyword-flexible recognition ('add', 'add task', 'add a task') reduces the precision burden on the user.

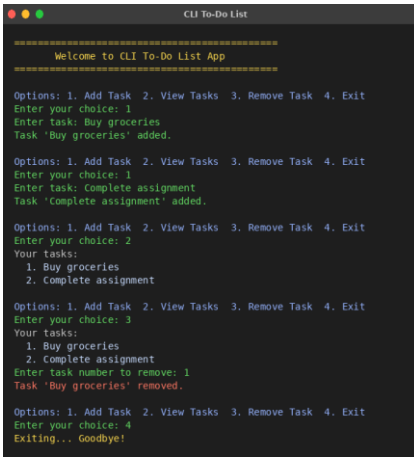
Sequential Task Dictation

- The total count is announced first ('You have 2 tasks') before individual tasks are read, giving a cognitive frame before detail.
- Tasks are spoken one at a time, preventing auditory overload from long continuous lists.

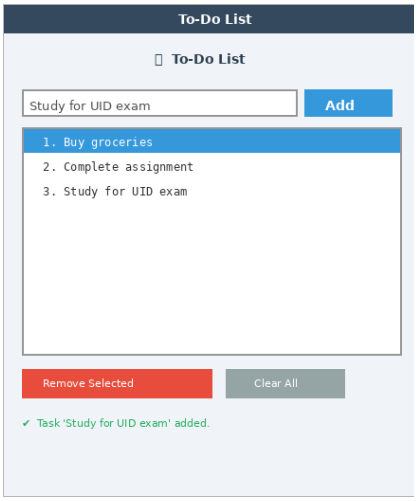
Error Recovery Chunking

- Each error type (timeout, unrecognised speech, API failure) produces a specific spoken message that reorients the user without restarting the loop.

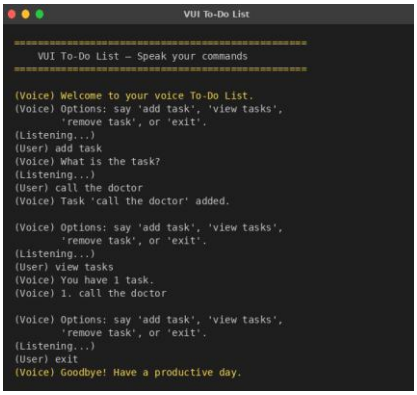
Screenshots of Executed Programs



CLI — Terminal Output



GUI — Tkinter Window



VUI — Voice Terminal

Result

All three To-Do List interfaces were successfully implemented and executed using Python. The CLI ran entirely in the terminal using built-in I/O. The GUI opened as a Tkinter desktop window with live task management. The VUI operated through spoken commands using SpeechRecognition and pyttsx3. Each interface demonstrated distinct interaction patterns, usability characteristics, and cognitive load profiles, confirming that interaction modality fundamentally shapes the user experience.